# ASTAS 3D Simulation Environment Guide

## 🎮 Gazebo-Style Simulation for ASTAS

This is a complete 3D simulation environment similar to Gazebo, built entirely in Python using:

- **PyBullet** - Physics engine (realistic collision, gravity, forces)
- **Panda3D** - 3D rendering engine (visualization, cameras)
- **PyQt5** - GUI control panel (spawn objects, configure sensors)

---

## 📦 Installation

```bash
# Core simulation engines
pip install pybullet panda3d PyQt5

# Additional dependencies
pip install numpy opencv-python

# Optional (for better performance)
pip install PyOpenGL PyOpenGL-accelerate

# ASTAS modules (if running integrated)
pip install ultralytics transformers torch
```

---

## 🚀 Quick Start

### Option 1: Headless Mode (No GUI)

```bash
```

```
# Run basic physics simulation
python simulation/sim3d_controller.py

# This will:
# - Initialize physics engine
# - Spawn test objects (person, vehicle, robot)
# - Run 10-second simulation
# - Print threat assessments every 5 seconds
```

### Option 2: GUI Mode (Full Interface)

```python
from simulation.sim3d_controller import ASTAS3DSimulation

# Create simulation with GUI
sim = ASTAS3DSimulation(use_gui=True, use_physics=True)

# Spawn objects
sim.spawn_object(ObjectType.PERSON, position=[5, 0, 1])
sim.spawn_object(ObjectType.VEHICLE, position=[-5, 5, 1])

# Run
sim.run_headless(duration=60.0, fps=30.0)
```

### Option 3: With Panda3D Rendering

```python
```

```python
# Full 3D visualization
from simulation.sim3d_env import RenderEngine, PhysicsEngine

# Initialize engines
physics = PhysicsEngine(gravity=-9.81)
renderer = RenderEngine()

# Create objects in both
physics_id = physics.create_capsule([0, 0, 1], radius=0.3, height=1.7)
render_id = renderer.create_capsule([0, 0, 1], radius=0.3, height=1.7)

# Synchronize in update loop
pos, orn = physics.get_position_orientation(physics_id)
renderer.update_object_pose(render_id, pos, orn)

# Run Panda3D main loop
renderer.run()
```

---

## 🎯 Example Scenarios

### Scenario 1: Border Patrol Simulation

```python
```

```python
from simulation.sim3d_controller import ASTAS3DSimulation, ObjectType

sim = ASTAS3DSimulation()

# Setup environment
# Create border zone (restricted area)
for x in range(-20, 21, 5):
    sim.spawn_object(ObjectType.OBSTACLE, [x, 10, 0.5])  # Fence

# Spawn patrol robot
robot = sim.spawn_object(ObjectType.ROBOT, [0, 0, 0.5])

# Spawn intruders at different locations
sim.spawn_object(ObjectType.PERSON, [5, 8, 0], velocity=[0, 0.5, 0])
sim.spawn_object(ObjectType.PERSON, [-8, 12, 0], velocity=[0, -0.3, 0])

# Run simulation
for i in range(1000):  # 1000 frames
    sim.step(dt=0.033)  # 30 FPS

    if i % 150 == 0:  # Every 5 seconds
        assessment = sim.run_astas_analysis()
        print(f"\n=== Time: {sim.simulation_time:.1f}s ===")
        print(f"Threat: {assessment['threat_level']}")
        print(f"Actions: {assessment['actions']}")
```

### Scenario 2: Building Security

```python
```

```python
sim = ASTAS3DSimulation()

# Create building layout
# Walls
wall_positions = [
    [10, 0, 1], [-10, 0, 1],   # North-South walls
    [0, 10, 1], [0, -10, 1]     # East-West walls
]
for pos in wall_positions:
    sim.spawn_object(ObjectType.OBSTACLE, pos)

# Entrance
entrance_camera = sim.spawn_object(ObjectType.CAMERA, [0, -11, 2])

# Spawn visitors
sim.spawn_object(ObjectType.PERSON, [0, -15, 0], velocity=[0, 1, 0])
sim.spawn_object(ObjectType.PERSON, [2, -15, 0], velocity=[0, 1, 0])

# Suspicious vehicle
sim.spawn_object(ObjectType.VEHICLE, [-20, 0, 1], velocity=[2, 0, 0])

# Enable all sensors
sim.sensor_data['camera']['enabled'] = True
sim.sensor_data['lidar']['enabled'] = True
sim.sensor_data['audio']['enabled'] = True

# Run
sim.run_headless(duration=30.0, fps=30.0)
```

### Scenario 3: Drone Surveillance

```python
```

```python
sim = ASTAS3DSimulation()

# Spawn surveillance drone
drone = sim.spawn_object(ObjectType.DRONE, [0, 0, 10])

# Create patrol pattern
patrol_points = [
    [10, 10, 10],
    [-10, 10, 10],
    [-10, -10, 10],
    [10, -10, 10]
]

# Spawn ground targets
for i in range(5):
    x = np.random.uniform(-15, 15)
    y = np.random.uniform(-15, 15)
    sim.spawn_object(ObjectType.PERSON, [x, y, 0])

# Simulate drone moving through patrol points
for i in range(len(patrol_points)):
    target = patrol_points[i]

    # Move drone toward target
    for step in range(100):
        # Simple movement (in real sim, use physics forces)
        current_pos = drone.position
        direction = np.array(target) - current_pos
        distance = np.linalg.norm(direction)

        if distance > 0.1:
            velocity = direction / distance * 2.0  # 2 m/s
            drone.position += velocity * 0.033

        sim.step(dt=0.033)

        # Check threats from aerial view
        if step % 30 == 0:
            assessment = sim.run_astas_analysis()
            if assessment['threat_score'] > 0.5:
                print(f"Drone Alert: {assessment['reasoning']}")
```

**Scenario 4: Multi-Sensor Fusion Test**

python

```python
sim = ASTAS3DSimulation()

# Place sensors at strategic locations
sensor_positions = [
    [10, 10, 2],   # Corner 1
    [-10, 10, 2],  # Corner 2
    [-10, -10, 2], # Corner 3
    [10, -10, 2]   # Corner 4
]

for pos in sensor_positions:
    sim.spawn_object(ObjectType.SENSOR, pos)

# Spawn moving targets
person1 = sim.spawn_object(ObjectType.PERSON, [0, 0, 0], velocity=[1, 0.5, 0])
person2 = sim.spawn_object(ObjectType.PERSON, [5, 5, 0], velocity=[-0.5, -1, 0])
vehicle = sim.spawn_object(ObjectType.VEHICLE, [-10, 0, 1], velocity=[2, 0, 0])

# Collect sensor data over time
sensor_log = []

for i in range(300):  # 10 seconds
    sim.step(dt=0.033)

    # Gather multi-sensor data
    if i % 30 == 0:
        data = {
            'time': sim.simulation_time,
            'camera': sim.sensor_data['camera']['data'],
            'lidar': len(sim.sensor_data['lidar']['data'].points) if sim.sensor_data['lidar']['data'] else 0,
            'objects': [(obj.object_type.value, obj.position.tolist()) for obj in sim.objects]
        }
        sensor_log.append(data)

# Analyze collected data
print("\nSensor Fusion Analysis:")
for entry in sensor_log:
    print(f"\nT={entry['time']:.1f}s:")
    print(f"  Camera detections: {len(entry['camera'])}")
    print(f"  LiDAR points: {entry['lidar']}")
    print(f"  Objects: {len(entry['objects'])}")
```

## 🎮 Interactive Controls

### Keyboard Controls (when using Panda3D renderer):

```
Camera:
  Arrow Keys - Rotate camera
  W/S - Move forward/backward
  A/D - Strafe left/right
  Q/E - Move up/down
  Mouse Scroll - Zoom in/out

Simulation:
  SPACE - Pause/Resume
  R - Reset simulation
  T - Run threat assessment
  1-5 - Spawn different objects
  ESC - Exit
```

### GUI Controls (PyQt5 panel):

```
Simulation Controls:
  [Start] [Stop] [Reset]
  Speed slider: 0.1x to 10x

Spawn Objects:
  Type dropdown: Person/Vehicle/Robot/Animal/Drone/Obstacle
  Position X/Y: -50 to +50
  [Spawn Object] button

Sensors:
  ☑ Camera
  ☑ LiDAR
  ☑ IMU
  ☐ Audio

Tabs:
  - Objects: Table of all spawned objects
  - Sensor Data: Real-time sensor readings
  - Threat Assessment: ASTAS analysis
  - Logs: Event log
```

## 🔧 Advanced Configuration

### Custom Physics Properties

```python
# Create object with custom physics
physics_id = sim.physics.create_box(
    position=[0, 0, 5],
    size=[1, 1, 1],
    mass=10.0,  # kg
    color=[1, 0, 0, 1]  # Red
)

# Apply force
import pybullet as p
p.applyExternalForce(
    physics_id,
    -1,  # Link index (-1 = base)
    [100, 0, 0],  # Force vector
    [0, 0, 0],  # Position
    p.WORLD_FRAME
)

# Set friction
p.changeDynamics(
    physics_id,
    -1,
    lateralFriction=0.8,
    spinningFriction=0.05,
    rollingFriction=0.01
)
```

### Custom Rendering

```python
```

```python
# Load custom 3D model (if available)
from panda3d.core import Loader

loader = Loader.getGlobalPtr()
model = loader.loadModel("path/to/model.egg")
model.reparentTo(render)
model.setPos(x, y, z)

# Custom materials
from panda3d.core import Material

mat = Material()
mat.setShininess(32.0)
mat.setAmbient((0.2, 0.2, 0.2, 1))
mat.setDiffuse((0.8, 0.8, 0.8, 1))
mat.setSpecular((1, 1, 1, 1))
model.setMaterial(mat)
```

### Sensor Configuration

```python
# Configure LiDAR parameters
sim.astas_lidar = LiDARProcessor(
    max_range=100.0,      # meters
    resolution=0.1,       # meters
    fov=360               # degrees
)

# Configure camera
sim.astas_detector = ObjectDetector(
    confidence_threshold=0.3,  # Lower = more detections
    iou_threshold=0.5
)

# Custom sensor update rate
sim.sensor_update_rate = 10  # Hz
```

---

## 📊 Data Collection & Analysis

### Record Simulation Data

```python
```

```python
import json

# Data recorder
class SimulationRecorder:
    def __init__(self, filename):
        self.filename = filename
        self.data = []

    def record_frame(self, sim):
        frame_data = {
            'time': sim.simulation_time,
            'frame': sim.frame_count,
            'objects': [
                {
                    'id': obj.object_id,
                    'type': obj.object_type.value,
                    'position': obj.position.tolist(),
                    'velocity': obj.velocity.tolist()
                }
                for obj in sim.objects
            ],
            'threat': sim.run_astas_analysis()
        }
        self.data.append(frame_data)

    def save(self):
        with open(self.filename, 'w') as f:
            json.dump(self.data, f, indent=2)

# Usage
recorder = SimulationRecorder('simulation_log.json')

for i in range(1000):
    sim.step(dt=0.033)

    if i % 10 == 0:  # Record every 10 frames
        recorder.record_frame(sim)

recorder.save()
```

### Replay Recorded Simulation

```python
```

```python
import json

with open('simulation_log.json', 'r') as f:
    recorded_data = json.load(f)

# Reset and replay
sim.reset()

for frame in recorded_data:
    # Restore object positions
    for obj_data in frame['objects']:
        obj_id = obj_data['id']
        pos = obj_data['position']

        # Find object and update
        for obj in sim.objects:
            if obj.object_id == obj_id:
                obj.position = np.array(pos)
                break

    # Display frame
    time.sleep(0.033)  # 30 FPS
```

---

## 🧪 Testing Scenarios

### Performance Benchmark

python

```python
import time

# Spawn many objects
for i in range(100):
    x = np.random.uniform(-20, 20)
    y = np.random.uniform(-20, 20)
    sim.spawn_object(ObjectType.PERSON, [x, y, 0])

# Measure FPS
start = time.time()
frames = 300

for i in range(frames):
    sim.step(dt=0.033)

elapsed = time.time() - start
fps = frames / elapsed

print(f"Performance: {fps:.1f} FPS with 100 objects")
```

### Stress Test

```python
# Maximum objects test
max_objects = 0

try:
    for i in range(1000):
        sim.spawn_object(ObjectType.OBSTACLE, [
            np.random.uniform(-50, 50),
            np.random.uniform(-50, 50),
            1
        ])
        max_objects += 1

        # Try to step simulation
        sim.step(dt=0.033)

except Exception as e:
    print(f"Max objects before crash: {max_objects}")
    print(f"Error: {e}")
```

---

## 📝 Tips & Best Practices

1. **Start Small**: Begin with 2-3 objects, then scale up
2. **Use Headless Mode**: For batch testing, disable rendering
3. **Record Sessions**: Save simulation data for analysis
4. **Sensor Calibration**: Adjust detection thresholds for your scenario
5. **Physics Tuning**: Adjust gravity, friction for realistic behavior
6. **Memory Management**: Remove objects when no longer needed
7. **Multi-threading**: Run physics and rendering in separate threads

---

## 🐛 Troubleshooting

### Issue: "PyBullet not installed"

```bash
pip install pybullet
```

### Issue: "Panda3D window not appearing"

```python
# Check if Panda3D is installed
pip install panda3d

# Try running basic Panda3D test
from direct.showbase.ShowBase import ShowBase
app = ShowBase()
app.run()
```

### Issue: "Physics simulation is slow"

```python
```

```python
# Reduce physics accuracy (faster, less accurate)
sim.physics.time_step = 1.0 / 60.0  # Was 1/240

# Reduce number of physics steps per frame
sim.update_physics(dt=0.033)  # Only step once
```

**Issue: "GUI not responding"**

```python
# Use QTimer for periodic updates
from PyQt5.QtCore import QTimer

timer = QTimer()
timer.timeout.connect(sim.step)
timer.start(33)  # 30 FPS
```

---

## 🎓 Learning Resources

- **PyBullet**: https://pybullet.org/
- **Panda3D**: https://www.panda3d.org/
- **PyQt5**: https://www.riverbankcomputing.com/software/pyqt/

---

## ✨ Next Steps

1. Add more object types (buildings, trees, etc.)
2. Implement advanced AI behaviors (patrol patterns, pursuit)
3. Add weather effects (rain, fog)
4. Create level editor
5. Export to Gazebo/ROS format
6. VR/AR integration

---

**Your complete 3D simulation environment is ready!** 🚀